STRIPE PROJECT

1. OLTP Data System

Annexes

1.C. OLTP Design Notes

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / OLTP Data System / Annexe C / v36 - 2026/11/05.*

1.C.1. Principes de conception

1.C.1.1. Fils conducteurs

Trois critères de décision qui reviennent systématiquement :

- ne jamais ralentir le chemin critique d'exécution lors d'une procédure de paiement (transaction),
  - découplage, écriture asynchrone, CDC vs. triggers ;
- préserver le cloisonnement des clients par marchand à tous les niveaux ;
- choisir la structure de données proportionnée à la richesse métier réellement portée par chaque information :
  - table quand il y a des métadonnées ou une logique d'état,
  - colonne simple sinon.

1.C.1.2. Principes structurants

Normalisation en 3NF

Le schéma élimine la redondance et garantit l'intégrité.
Chaque fait vit à un seul endroit, ce qui évite les incohérences lors des écritures massives.

Clés primaires en UUID

Génération distribuée sans point de contention central

- indispensable pour le futur *sharding* d'une table qui encaisse des millions d'écritures par jour.

Les UUID v7, ordonnés dans le temps, gardent les insertions groupées dans les index. Cela compense en partie l'absence de partitionnement.

`Transaction` **est une entité *auto-suffisante***

Montant, devise et date permettent toute conversion *a posteriori* côté analytique.

Ce choix évite d'introduire une table de taux de change OLTP et réoriente la conversion des devises dans le domaine OLAP (avec un taux de référence figé, renouvelable périodiquement, pour permettre la comparaison des montants monétaires).

1.C.1.3. Découplage autour de la transaction

`Refund` , `Chargeback` , `FraudScore` **sont sortis de** `Transaction`

Raisons : il s'agit, d'une part, d'éviter des colonnes `NULL` massives sur une table à fort volume (la plupart des transactions n'ont ni remboursement ni litige), d'autre part, de ne pas verrouiller la table transactionnelle avec des écritures tardives ou asynchrones (un score de fraude s'écrit en temps réel par un service séparé, une rétro-facturation évolue sur plusieurs semaines).

`Refund` **vs** `Chargeback`

Ces entités reflètent deux réalités opposées : le remboursement est une décision du marchand,
la rétro-facturation une contestation imposée par la chaîne bancaire. Attributs, cycles de vie et acteurs diffèrent entièrement, d'où l'absence de mutualisation.

`FraudRiskLevel`

Dans un système de fraude réel, le niveau de risque déclenche une action différenciée (auto-approbation, revue manuelle, blocage). `FraudRiskLevel` porte les attributs booléens `requires_manual_review` et `blocks_transaction` pour permettre une dérivation de l'action à prendre au lieu de la coder en dur côté application à chaque occurrence de `high`). Une simple énumération `risk_level` (`low` | `medium` | `high`) ne permet pas de faire ça.

Séparation de `FraudReview` et `FraudDecision` de `Transaction`

Même logique de découplage que pour `FraudScore` : une revue humaine est un événement asynchrone, déclenché parfois plusieurs jours après la transaction (échantillonnage, contestation), et une décision peut être révisée sans jamais toucher à la ligne `Transaction`. Séparer ces deux entités évite qu'une confirmation de revue - écriture différée, potentiellement lente si elle implique un processus humain - ne verrouille une table à fort volume et à forte contrainte de latence.

1.C.1.4. Cloisonnement Merchant / Customer

`Merchant` comme entité distincte de `Customer`

Le marchand porte un rôle réglementé - celui qui encaisse assume KYC/KYB, responsabilité en cas de litige, reversement des fonds - que le client n'a pas.

`Customer` modélise la relation client-marchand et non la personne physique

Une même personne achetant chez trois marchands produit trois lignes distinctes. Ce n'est pas un défaut mais une conséquence voulue de la confidentialité entre marchands concurrents, du RGPD (le consentement est donné à un marchand précis) et de la responsabilité de traitement. Le rapprochement transverse d'une identité, quand il est nécessaire, se fait dans une couche séparée (NoSQL), pas dans l'OLTP.

1.C.1.5. Gestion des énumérés

Tables de référence pour les énumérés porteurs de métadonnées

Tables concernées : `TransactionStatus`, `MerchantStatus`, `SubscriptionStatus`, `ChargebackStatus`, `ChargebackReasonCode`, `PaymentMethodType`, `CardBrand`.

Critère retenu : dès qu'une valeur porte une logique métier exploitable (`is_terminal`, `allows_refund`, `can_process_payments`, `triggers_dunning`) ou que la liste est vivante (codes de rétro-facturation normalisés par les réseaux, nouvelles marques de carte), une table s'impose.

Ajouter une valeur se traduit par un `INSERT` au lieu d'une migration de schéma.

Clé primaire naturelle et lisible (`'refunded'` non entière).

La colonne référençante reste interprétable sans jointure ; la jointure ne sert qu'à obtenir les métadonnées enrichies.

Contrainte CHECK conservée pour les listes courtes et figées

Une table dédiée serait disproportionnée pour ces liste sans métadonnée à porter.
Listes concernées :

- `Transaction.device_type` (mobile | desktop | tablet) :
  purement descriptif ; aucune note, aucune règle du MCD ni du modèle physique ne décrit une action différente selon la valeur.
- `Subscription.billing_cycle` (weekly | monthly | yearly) :
  même raison ; la seule métadonnée référencable, nombre de jours par cycle, est calculée côté application au moment du prélèvement.
- `ConsentRecord.purpose` (marketing | analytics | data_sharing | profiling)
  et `.source` (checkout | email | account_settings) : purement déclaratives ;
  qualifient l'enregistrement sans déclencher de comportement différent ;
  c'est `is_granted` qui porte la logique d'état).
- `ConsentRecord.legal_basis` (consent | contract | legitimate_interest) :
  cas limite ; le modèle ne porte actuellement aucune règle qui permettrait de distinguer le traitement selon cette valeur ; à surveiller.
- `DataSubjectRequest.request_type` (access | rectification | erasure | portability | objection) :
  chaque type a un traitement métier différent, mais cette différence vit dans le code applicatif, pas dans une donnée que le schéma aurait besoin de porter.
- `SecurityIncident.severity` (low | medium | high | critical)
  et `.incident_type` (unauthorized_access | data_breach | anomalous_activity | failed_auth_burst) :
  passage en référence plausible (escalade automatique, notification différenciée) mais non spécifié dans le cahier des charges.

1.C.1.6. Abonnements et Produits

Modélisation de l'abonnement comme entité

`Subscription` porte un cycle de vie propre (actif, en pause, en impayé (`past_due` , avec relance automatique), résilié) et relie client, produit et moyen de paiement autorisé pour le prélèvement récurrent.

Nullabilité de `Transaction.product_id` et `Transaction.subscription_id`

`Transaction.product_id` et `Transaction.subscription_id` doivent être nullables pour deux raisons :

- Permettre les transactions suivantes :
  - Achat ponctuel d'un produit du catalogue --> `product_id` renseigné, `subscription_id` nul.
  - Prélèvement récurrent généré par un abonnement --> `subscription_id` renseigné, `product_id` renseigné (car l'abonnement porte lui-même un produit).
  - Paiement libre, hors catalogue --> `product_id` et `subscription_id` sont nuls (cf. MCD/règle #6).
- Permettre que les indicateurs de suivi de performance produit (revenu total, nombre de ventes, popularité par produit ; cf. Business Case/ Data Source DS2 : "Product Performance Metrics") soient calculés par des agrégations simples du type " `SELECT product_id, SUM(amount)FROM Transaction GROUP BY product_id` ".
  Pour que cette requête soit à la fois directe (pas de jointure conditionnelle compliquée) et correcte (le total par produit reflète vraiment ce produit, ni plus ni moins),
  il faut que `product_id` soit renseigné directement sur `Transaction` pour toutes les ventes qui concernent un produit et nul pour celles ce qui n'en concernent aucun.

`Transaction.is_merchant_initiated`

Un prélèvement d'abonnement n'a ni clic, ni appareil, ni géolocalisation exploitables :
ce champ évite qu'un modèle de fraude calibré sur le comportement client ne signale systématiquement les paiements récurrents comme suspects.

1.C.1.7. Sécurité et conformité

Enregistrements des journaux en écriture seule

Les tables d'enregistrements d'évènement destinés aux audits de sécurité ou de conformité sont en écriture seule (suppression des droits sur `UPDATE` ni `DELETE`).

Tables concernées : `DataAccessLog` , `ChangeAuditLog` , `FraudScoringLog` , `FraudScore` , `ConsentRecord` .

Écriture asynchrone des journaux

Pour que l'enregistrement des journaux ne ralentissent pas les paiements, en doublant le coût de chaque écriture sur `Transaction` , les tables `DataAccessLog` et `ChangeAuditLog` doivent être écrites par des mécanismes asynchrones, qui n'utilisent pas les déclencheurs TRIGGER (les *triggers* se déclenchent dans la même transaction que l'écriture d'origine).

`ChangeAuditLog` sera alimentée par un mécanique CDC (Change Data Capture), qui utilise un outil externe (ex: Debezium) pour lire le WAL (write-ahead log) du SGBD (PostgreSQL) *a posteriori*.

Un autre mécanisme asynchrone devra être mis en place pour tracer les *consultations* dans `DataAccessLog` (par exemple, en publiant les évènements d'accès sur un bus d'évènements externe).

Double rattachement de `ConsentRecord` au client et au marchand

`ConsentRecord` est également rattaché au marchand car un client *consent* à un marchand précis et pas à l'intermédiaire *Stripe*.

Délais matérialisés comme données mesurables

Propriétés concernées :

- `DataSubjectRequest.due_at` / `fulfilled_at` ,
- `SecurityIncident.detected_at` / `notified_at` .

Un rapport de conformité doit prouver le respect des délais légaux et pas seulement l'existence d'une procédure.

1.C.1.8. Gestion des moyens de paiement : jeton et empreinte

Cette section expose les éléments justificatifs de l'introduction des informations "*jeton de paiement*" (`token_reference`) et "*empreinte de carte*" (`fingeprint`) dans la table `PaymentMethod` .

Origine du besoin

On attend de la table `PaymentMethod` qu'elle conserve au moins les informations qui doivent être affichées ou vérifiées sans risque : type, marque, quatre derniers chiffres (`card_last4`) et date d'expiration. Cette exigence permet de respecter l'interdiction d'afficher le PAN en clair (PCI-DSS, cf. §1.C.3) mais n'est pas suffisant pour satisfaire les besoins suivants :

- **débiter** : un prélèvement, en particulier récurrent et à l'initiative du marchand, nécessite de présenter au réseau une référence au moyen de paiement (la marque et quatre derniers chiffres du numéro de carte ne suffisent pas) ;
- **reconnaître** : les processus de détection de fraude ont besoin de détecter qu'une même carte est utilisée par plusieurs clients ou plusieurs marchands en se référant à un identifiant unique de la carte (là encore, marque et quatre derniers chiffres ne sont pas suffisant).

`token_reference` : la référence de jeton qui permet de débiter

`token_reference` est la référence opaque au moyen de paiement, délivrée par le coffre de tokens ou par le réseau de cartes en amont du traitement de la transaction par Stripe. Le numéro de carte (PAN) n'entre jamais dans le système makis celui-ci peut stocker les jetons de paiement.

Caractéristiques :

- non nul : un moyen de paiement sans jeton de paiement ne peut pas être débité ;
- unique : chaque instance moyen de paiement est caractérisée par un unique jeton (avec la non-nullabilité du jeton, cela en fait une clé d'identification alternative du moyen de paiement).
- donnée sensible : c'est une donnée personnelle du point de vue du RGPD, portée seulement par `PaymentMethod` , qui ne doit jamais être utilisée comme clé étrangère, et qui doit être pseudonymisable pour l'utilisateur qui exerce son droit à l'effacement (*RGPD*) (elle ne peut pas être effacée puisqu'elle n'est pas nullable).

`fingerprint` : l'empreinte qui permet de reconnaître la carte

`fingerprint` est une empreinte *salée* et *non réversible* de la carte, calculée au moment de la tokenisation, dans la zone qui voit le PAN. Une même carte physique donne la même empreinte, alors que chaque enregistrement reçoit un `token_reference` différent. Une empreinte seule ne permet pas de débiter (c'est le jeton de paiement qui permet ça). Usages : dédoublonnage des cartes d'un client, mesure de la vélocité d'une carte entre marchands, calcul de features basé sur la carte pour le modèle de fraude.

Caractéristiques :

- optionnelle : tous les moyens de paiement ne sont pas des cartes.
- non unique : la même carte peut apparaitre légitimement chez plusieurs clients et plusieurs marchands.
- donnée sensible : comme le jeton de paiement, c'est une donnée personnelle. Elle doit être effacable (nullifiée) et/ou pseudonymisable (avant d'être inscrite dans une table de journaux non modifiable).

La clé primaire de la table `PaymentMethod` reste `payment_method_id`

`token_ref` est unique, mais il ne peut pas servir d'identité au modèle :

- il **change** (renouvellement du jeton, mise à jour automatique de carte, changement de fournisseur) et il est volontairement écrasé à l'effacement ; une clé primaire doit rester stable ;
- il est **sensible** : une clé primaire est recopiée dans les clés étrangères (`Transaction` , `Subscription`), dans `ChangeAuditLog` et dans le flux CDC ; en faire la clé étendrait la diffusion d'une donnée qui permet de débiter ;
- il est **volumineux et externe** (`varchar(255)` , format imposé par un tiers) alors que `payment_method_id` est un `uuid` interne, compact, sur lequel s'appuient les clés étrangères composites de la table la plus écrite du système.

L'unicité de `token_ref` est donc une contrainte d'intégrité, et `payment_method_id` l'identité stable.

Périmètre

- Le coffre de tokens et le calcul de l'empreinte sont exclus du périmètre du modèle *OLTP* :
  - Le modèle suppose un composant de tokenisation qui calcule l'empreinte avec un sel secret et constant. Ce composant est à décrire dans le plan de sécurité (D6) et dans l'architecture des pipelines de données (D5).
- Le sel de l'empreinte de carte doit rester secret :
  - sa divulgation rendrait les empreintes attaquables par force brute, l'espace des numéros de carte étant restreint. Un sel qui change romprait la reconnaissance d'une même carte.
- L'inscription des valeurs dans les journaux d'accès ou d'écriture doit être :
  - Le flux CDC qui alimente `ChangeAuditLog` copie les valeurs « avant / après » des colonnes modifiées : sa transformation doit **masquer `token_reference` et `fingerprint`** dans `changed_fields` . Sinon le journal, immuable, conserverait ces valeurs et l'effacement RGPD serait contourné (cf. 02_complement.sql, §9). Cette règle n'est pas encore spécifiée.

- Application aux autres moyens de paiement :
  - En principe, pour les moyens de paiement qui ne sont pas des cartes (virement, portefeuille, prélèvement), `token_reference` désigne la référence correspondante chez le prestataire (mandat, jeton de portefeuille). L'hypothèse reste à confirmer.

Couverture des exigences

#	Exigence	Réponse
BR1	traiter les paiements	`token_reference` rend le prélèvement possible sans stocker le PAN
BR5	conformité PCI-DSS, RGPD	pas de PAN + pseudonymisation de `token_reference` et de `fingerprint` à l'effacement
TR5	idem	idem

1.C.1.9. Garantie de l'idempotence d'une transaction

Le client ou le réseau peut avoir besoin rejouer une requête de paiement pour de multiples raisons valables (délai dépassé, réponse perdue). Le système ne doit donc veiller à ne jamais créer la même transactions deux fois (ce qui conduirait le système à débiter deux fois pour la même intention de paiement).

L'identifiant de transaction ne suffit pas

- `transaction_id` est un objet de type `uuid` *fabriqué par l'application à chaque tentative*, avant l'insertion de la transaction dans sa table. Avec ce fonctionnement , *une requête rejouée est une nouvelle requête* : elle reçoit un nouvel identifiant, et l'insertion ne rencontre aucun conflit.
- Détecter un rejeu implique une valeur *fournie par l'appelant* et *identique d'une tentative de paiement à l'autre* : c'est la clé d'idempotence (cf. en-tête `Idempotency-Key`).

En-tête `Idempotency-Key` :

Un en-tête HTTP est une ligne de métadonnées que le client ajoute à une requête, à côté du corps (contenu du paiement au format JSON). `Idempotency-Key` est un en-tête de ce type, utilisé par les API de paiement pour rendre une requête rejouable sans danger.

Mécanisme de création d'une transaction avec idempotence :

La table `IdempotencyKey (merchant_id, idempotency_key, transaction_id, created_at)` a pour clé primaire (PK) la clé composite unique `(merchant_id, idempotency_key)` . La procédure de création (atomique) est la suivante :

Etape 1 : réserver la clé d'idempotence.

```
INSERT IdempotencyKey(merchant_id, idempotency_key, transaction_id, created_at)
ON CONFLICT (merchant_id, idempotency_key) DO NOTHING RETURNING transaction_id
```

Si aucune ligne n'est retournée, la requête est un rejeu : l'*application* abandonne, relit la transaction déjà associée à la clé et renvoie son résultat. L'identifiant fraîchement généré est jeté.

Etape 2 : insérer la transaction avec le même identifiant de transaction qui a servi à créer la clé.
Si aucune ligne n'est insérée (pas de plan tarifaire en vigueur, ou pas de frais fixe pour la devise), l'application annule toute la transaction SQL, clé réservée comprise.

- La clé étant insérée avant l'insertion de la' transaction, la clé étrangère `IdempotencyKey.transaction_id` est `DEFERRABLE INITIALLY DEFERRED` : elle n'est vérifiée qu'au `COMMIT` de la transaction SQL, alors que l'insertion de la transaction a déjà été faite. Si l'étape 2 échoue sans annulation explicite, le `COMMIT` échoue aussi.

Que se passe-t-il dans le cas de deux requêtes identiques simultanées : la seconde attend, au niveau de l'index unique, que la première valide, puis trouve la clé prise et ne fait rien.

Unicité de l'identifiant de transaction

L'unicité de l'identifiant `transaction_id` est garantie par la clé primaire de `Transaction` (index unique), quelle que soit la façon dont l'identifiant est fabriqué. `uuidv7()` ne fait que rendre une collision négligeable. Si elle se produisait, l'étape d'insertion de la transaction (cf. sous-section précédente) échouerait sur la clé primaire et l'application réessaierait avec un autre identifiant, sans corruption de données.

Pourquoi la clé d'idempotence est-elle enregistrée dans une table séparée?

- **Cloisonnement** (cf. §1.C.1.4) : la clé est propre à un marchand. Si le client choisissait la clé primaire de `Transaction` , un marchand pourrait provoquer un conflit avec la transaction d'un autre marchand, ou apprendre qu'elle existe ;

- **Format libre** : la clé est un texte (jusqu'à 255 caractères) tel que les marchands l'envoient déjà, alors que `transaction_id` reste un `uuid` interne ;
- Durée de vie courte** : la clé n'est utile que pendant la fenêtre de rejeu (de 24 h à 30 jours selon le choix retenu). Elle se purge (index sur `created_at`) sans toucher à `Transaction`, la table la plus écrite et conservée longtemps ;
- **Résultat exploitable** : la table donne directement la transaction associée à un premier essai, alors qu'un simple conflit sur `Transaction` ne dirait rien de son issue.

Limite :

Le contenu d'un rejeu n'est pas comparé à celui du premier appel (montant, moyen de paiement). Une même clé rejouée avec un montant différent sera donc traitée comme le rejeu d'une même transaction. On ignore ce cas de figure dans le périmètre de l'exercice pour ne pas compliquer encore le processus de création des transactions (il faudrait stocker une empreinte de la requête dans `IdempotencyKey`).

1.C.2. Justification du modèle de tarification

Origine du besoin

La paire de tables (`PricingPlan`, `PricingPlanFee`) capture la commission *Stripe* à la source pour permettre a posteriori le calcul des montants des commissions dans la devise de référence dans le modèle analytique (cf. Business Requirement BR2 : "revenue analysis", et Deliverable D8 : requêtes de revenu).

Pourquoi a-t-on besoin de deux tables?

`commission_rate` (un taux) et `fixed_fee` (un montant monétaire, dans une devise donnée) n'ont pas la même relation à la devise : le taux, à l'inverse du montant de frais fixe, est indépendant de la devise.

Modéliser `fixed_fee` comme attribut scalaire de `PricingPlan` suppose que chaque plan à sa propre devise, hypothèse qui devient fausse dès qu'un marchand traite plusieurs devises.
`PricingPlanFee` porte donc `fixed_fee` comme propriété d'une relation *n,n* entre `PricingPlan` et `Currency` : chaque plan peut être libellé dans plusieurs devises, chacune avec son propre frais fixe, sans conversion de change nécessaire pour l'appliquer.

Pourquoi `PricingPlan` est-il versionné dans le temps?

`effective_from` / `effective_to` permettent à un marchand de renégocier sa commission
(ou de changer de catégorie de risque) sans réécrire l'historique :
les transactions passées restent rattachées au plan qui était réellement en vigueur à leur date.
Cette contrainte n'est pas exprimable dans le modèle physique
et doit être transcrite en contrainte applicative (ou en trigger) comme l'absence de chevauchement des plages `[effective_from, effective_to)` pour un même `merchant_id`.

Pourquoi `Transaction` porte-t-il à la fois `pricing_plan_id` et `fee_amount`, alors que `fee_amount` est dérivable de `pricing_plan_id` ?

Il s'agit de permettre à cette propriété d'être recalculable tout en traçant chaque valeur calculée pour figer l'historique des valeurs.
En particulier, lorsqu'un plan tarifaire est corrigé rétroactivement, un écart entre la valeur stockée et la valeur recalculée
devient lui-même un signal exploitable (audit, détection d'anomalie de facturation).

Contraintes de cohérence non exprimables dans le modèle physique

Pour une transaction donnée :

- le plan référencé par `pricing_plan_id` doit bénéficier au marchand de cette même transaction ==> (`Transaction.merchant_id == PricingPlan[Transaction.pricing_plan_id].merchant_id`);
- la valeur de `fee_amount` doit rester calculable à partir des lignes de `PricingPlan` et de `PricingPlanFee` correspondant à la transaction :
  - fee_amount[transaction.currency] = princing_plan_fee[princing_plan[transaction]; transaction.currency].fixed_fee + princing_plan[transaction].commission_rate * transaction.amount

---

1.C.3. Justification du modèle de sécurité et conformité

Origine du besoin

Les exigences *BR5* (Business Requirement) et *TR5* (Technical Requirement) exigent explicitement les fonctionalités suivantes :

- journalisation d'audit,
- conformité RGPD/PCI-DSS,
- et reporting de conformité automatisé.

Ces fonctions sont réalisées par les tables suivantes :
`DataAccessLog`, `ChangeAuditLog`, `FraudScoringLog`, `FraudScore`, `FraudReview`, `FraudDecision`, `ConsentRecord`, `DataSubjectRequest` et `SecurityIncident`.

Tables de référence `AuditActorType` et `AuditAction`

Pourquoi `AuditActorType` et `AuditAction` sont-elles des tables de référence, et non de simples `CHECK` sur des listes de valeurs figées?
Ces tables portent une logique métier qu'une contrainte déclarative ne peut pas exprimer :

- `is_human` distingue un accès humain d'un accès automatisé (pertinent pour la détection d'anomalie),
- `is_sensitive` et `requires_justification` alimentent l'alerting temps réel et l'obligation de motif.

Séparation des journaux `DataAccessLog` et `ChangeAuditLog`

Pourquoi `DataAccessLog` (lecture) et `ChangeAuditLog` (écriture) sont-elles deux tables distinctes? Ces tables portent de nombreux attributs différents et les fusionner produirait des colonnes NULL à grande échelle (`Refund`, `Chargeback`, `FraudScore` sont séparés de la table `Transaction` pour la même raison). La fusion de ces table reste légitime dans le modèle analytique (reporting consolidé).

Pourquoi les tables de journalisation sont-elles en écriture seule (append-only)

Un journal modifiable ne prouve rien. En conséquence, les droits `UPDATE` et `DELETE` sont supprimés pour tout le monde, y compris pour les administrateurs, sur les tables de journaux : `FraudScoringLog`, `DataAccessLog`, `ChangeAuditLog`.

La table `ConsentRecord` suit le même principe mais pour une raison réglementaire différente : un retrait de consentement doit créer une nouvelle ligne, et ne pas écraser la précédente, car on doit conserver un historique complet des décisions du client.

Pourquoi les tables d'évènements `DataSubjectRequest` et `SecurityIncident` ne sont-elles pas en écriture seule?

Ce sont des entités de *workflow* dont le statut évolue légitimement dans le temps (`received` --> `fulfilled`, `open` --> `closed`).
Les dates de traitement complètent l'enregistrement initial et évitent de créer une nouvelle ligne à chaque étape du workflow:

- `due_at` encode directement l'échéance légale RGPD (un mois),
- l'écart entre `detected_at` et `notified_at` est le point de contrôle direct du délai RGPD de notification (72 heures).

Nullabilité de `merchant_id` ?

Il n'y a pas de marchand à référencer dans `DataAccessLog`, `ChangeAuditLog`, et `SecurityIncident`, lorsqu'un accès ou un incident est transverse à la plateforme,
et indépendant d'un marchand précis. À l'inverse, `merchant_id` ne doit pas être nul dans `ConsentRecord` et `DataSubjectRequest` car un client consent toujours à un marchand précis.

Articulation avec le reste du schéma

Le fait de tracer les 4 derniers chiffres du PAN dans `PaymentMethod.card_last4` participe également aux exigences de conformité avec PCI-DSS.

Tables de référence `Regulation`, `DSR Status` & `SI Status`

En application des principes de gestion des énumérés (cf. §1.C.1.5) :

- `DataSubjectRequestStatus` modélise un statut de workflow avec *état terminal*(`fulfilled`, `rejected` ou `closed`) - comme `is_terminal` dans `Transaction Status`, `Subscription Status` et `Chargeback Status`.
- Même principe de modélisation pour `SecurityIncidentStatus`.
- `Regulation.response_deadline_days` représente un délai légal qui prend une valeur différente en fonction de la réglementation *GDPR*/*CCPA*.
    - `DataSubjectRequest.due_at` devient dérivable de `received_at` + le délai légal.

1.C.4. Passage à l'échelle : indexation, partitionnement, sharding

Origine du besoin

TR3 [D0] demande de « mettre en œuvre des stratégies d'indexation, de partitionnement et de mise en cache » et d'« envisager » bases distribuées et sharding. Un modèle de données ne peut y répond qu'en partie : indexation, choix de clés, découpage des entités.

Indexation

Les index sont définis dans [D2-B]. Les requêtes de lot s'appuient sur des index partiels à définir (`ix_subscription_due` , `ix_transaction_open_status` , `ix_dal_denied`). Les balayages temporels (par ex. `Transaction(created_at)` , `DataAccessLog(occurred_at)`) et la purge de rétention s'appuient sur des index sur la colonne de date.

Partitionnement

- Le partitionnement mensuel de la table des transactions complexifie le modèle de données transactionnel au delà de la limite acceptable dans le cadre de l'exercice [D0].
- En effet, la clé primaire d'une table partitionnée est composée de la clé primaire simple de la table non partitionnée et d'une clé de partitionnement additionnelle, la clé dite *de partition*. Ainsi, la clé primaire de la table `Transaction` *partitionnée par mois* devient `(transaction_id, created_at)` (au lieu de `transaction_id` lorsque la table n'est pas partitionnée). Les tables filles qui référencent `Transaction` *partitionnée* doivent donc porter `transaction_created_at` en plus de `transaction_id` .
- > L'abandon du partitionnement des transactions est donc purement pédagogique, le modèle de données transactionnel étant jugé déjà suffisamment complexe sans partitionnement au vu des besoins de l'exercice [D0].

Sharding

- Le modèle prépare un sharding sans le mettre en œuvre : clés UUID générées sans point central, cloisonnement des données par marchand. La clé de répartition naturelle serait `hash(merchant_id)` . La stratégie de répartition n'est pas formalisée.

Mise en cache et architecture distribuée

Elles ne relèvent pas du schéma de données (cf. [D0] §0.4.3).

Références

Spécifications

- [D0] Stripe Project Introduction

Annexes

- [D2-B] OLTP Physical Model